

00-零基础期末总复习资料

目录

1	软件工程与计算 II 零基础期末突击总复习资料	2
---	-------------------------	---

1 软件工程与计算 II 零基础期末突击总复习资料

1.1 使用方法

这份资料只服务一个目标：在期末大题中稳定写出步骤分。往年卷完整结构以 Exam/2020.pdf、Exam/2022.pdf 为主，2025 卷加入了 Spring Boot 三层代码、AI 情境、排课系统和代码改造题。Review 资料补充了旧题中的过程模型、质量保障、配置管理、HCI、测试与维护。

推荐顺序：

先背 10 类题型模板

再做 8 套模拟卷

最后回看现有 00-05 分卷讲义中的细节

1.2 一、考试题型蓝图

稳定题型如下：

1. 简答：软件工程、生命周期、CI、SCM、再工程、过程模型。
2. 需求：需求层次、需求类型、用例、用户故事、SRS。
3. 分析模型：概念类图、系统顺序图、状态图。
4. 体系结构：Web 分层物理包图、REST、Spring Boot 三层职责、接口声明。
5. 详细设计：Controller-Service-Repository 顺序图、控制风格、GRASP。
6. 模块化：耦合、内聚、信息隐藏、包循环依赖、SOLID。
7. 设计模式：Strategy、Abstract Factory、Factory Method、Singleton、Iterator、Observer。
8. 软件构造：代码坏味道、表驱动、重构、防御式编程、契约式设计。
9. 测试：等价类、边界值、条件覆盖、分支覆盖、圈复杂度、Mock。
10. HCI：可用性、Nielsen 启发式、Fitts 定律、反馈、导航、一致性。

十题结构来自 2020、2022 的完整卷面。2025 的变化是业务场景改成教师排课系统，题目更偏代码片段和工程判断，但核心考点没有脱离上述结构。

1.3 二、简答题速背

1.3.1 软件工程

标准答案：

软件工程是把系统化、规范化、可量化的方法应用于软件开发、运行和维护，并研究这些方法的学科。

展开句：

软件工程关注的不只是编码，还包括需求、设计、构造、测试、维护、配置管理、质量保障和项目管理，使多人协作的软件

1.3.2 软件生命周期

答案模板：

软件生命周期是软件从提出、开发、运行、维护到退役的整个过程。
常见模型包括瀑布模型、增量迭代模型、演化模型、螺旋模型。

1.3.3 增量迭代模型与演化模型

增量迭代模型：把系统分成多个增量，每次交付一部分可运行功能，并通过多轮迭代完善。
演化模型：先交付初始版本或原型，再根据用户反馈和环境变化持续演化。
区别：增量迭代强调按功能批次交付，演化模型强调需求理解和系统形态随反馈变化。
共同点：都不是一次性完成，均支持反馈和逐步完善。

1.3.4 配置管理与变更控制

配置管理活动：

配置项识别、版本管理、变更控制、配置审计、状态报告、发布管理。

变更控制过程：

1. 提交变更请求。
2. 分析影响和成本。
3. 批准或拒绝变更。
4. 实施变更。
5. 验证变更结果。
6. 更新配置状态和相关文档。

1.3.5 持续集成

持续集成是开发者频繁把代码合并到主干，由自动化构建、自动化测试和质量检查及时发现集成问题。
好处：降低集成风险，尽早发现缺陷，缩短反馈周期，提升版本可发布性。
工具链：Git、GitHub/GitLab、CI 服务、构建工具、单元测试、静态分析、制品仓库、部署脚本。

1.3.6 正向工程、逆向工程、再工程

正向工程：从需求和设计出发构造新系统。
逆向工程：从已有代码或系统中恢复设计、需求或业务规则。
再工程：在理解旧系统的基础上重构、迁移或改造，使其更易维护或适配新环境。

1.4 三、需求题模板

1.4.1 需求层次

业务需求：为什么建设系统，体现组织目标和业务价值。

用户需求：用户希望通过系统完成什么任务。

系统级需求：系统必须提供的可验证行为或约束。

例子：校园卡充值。

业务需求：降低人工窗口充值压力，提高自助服务比例。

用户需求：学生希望在网页端自助为校园卡充值并查看记录。

系统级需求：系统接收充值请求后校验卡状态，支付成功后更新余额并生成充值记录。

1.4.2 需求类型

功能需求：系统要做什么。

性能需求：速度、容量、吞吐量、响应时间。

质量属性：安全性、可靠性、可用性、可维护性、易用性。

对外接口：与其他系统、设备、用户界面的交互接口。

约束：技术栈、法规、部署环境、业务规则。

数据需求：需要保存的数据字段、关系、完整性规则。

判断句式：

该需求属于功能需求，因为它描述系统在某个输入或事件下必须执行的业务行为。

该需求属于数据需求，因为它约束了数据字段格式、取值范围或持久化内容。

该需求属于对外接口，因为它规定本系统与外部系统的调用方式、数据格式或异常处理。

1.4.3 从用户需求到系统级需求

1. 明确用户目标和参与者。
2. 划定系统边界。
3. 拆分正常流程和异常流程。
4. 识别输入、输出、数据对象和业务规则。
5. 写成可验证的系统响应。
6. 补充非功能需求、接口和约束。
7. 通过评审和测试用例验证需求。

1.5 四、用例与分析模型

1.5.1 用例图四要素

参与者：系统外部与系统交互的人、外部系统、设备或定时器。

用例：系统为参与者提供的有价值服务。

系统边界：区分系统内外。

关系：关联、include、extend、泛化。

include 与 extend：

include：每次都要执行的公共行为，基本用例指向被包含用例。

extend：特定条件下发生的可选或异常行为，扩展用例指向基本用例。

1.5.2 概念类图步骤

1. 从题干中识别问题域名词。
2. 删除界面控件、技术实现类和动词。
3. 保留有状态、有业务意义、有关系的概念类。
4. 识别关联、聚合、组合、继承。
5. 加多重性。
6. 添加关键属性。

概念类图答题格式：

```

类：User, CampusCard, RechargeRecord, PaymentOrder
关联：User 1 -- 0..* CampusCard
关联：CampusCard 1 -- 0..* RechargeRecord
组合：RechargeRecord 1 -- 1 PaymentOrder
属性：CampusCard(cardNo, balance, status)

```

1.5.3 系统顺序图

```

参与者 -> 系统：系统事件
系统 -> 外部系统：外部调用
系统 --> 参与者：返回结果

```

系统顺序图只把系统当成黑盒，不画 Controller、Service、Repository。

1.6 五、体系结构题模板

1.6.1 Web 物理包图

固定结构：

```

客户端 / 浏览器
html
css
javascript
presentation 展示层
|
| HTTP + REST API + JSON
v
服务器端
controller/api
service 逻辑层
repository 数据层接口
domain/entity
database

```

答题要点：客户端必须出现 html、css、javascript 和展示层；服务器端必须出现逻辑层和数据层；跨网络连线标注 HTTP、REST API、JSON。

1.6.2 Spring Boot 三层职责

Controller: 接收 HTTP 请求, 参数校验, 调用 **Service**, 返回 DTO 或 JSON。

Service: 执行业务流程、事务控制、业务规则、协调领域对象和外部服务。

Repository: 封装数据访问, 提供查询和保存方法, 不写业务流程。

1.6.3 接口代码模板

Service:

```
package edu.nju.demo.service;

public interface RechargeService {
    RechargeResult recharge(RechargeCommand command);
}
```

Repository:

```
package edu.nju.demo.repository;

import java.util.Optional;

public interface CampusCardRepository {
    Optional<CampusCard> findByCardNo(String cardNo);
    void save(CampusCard card);
}
```

Controller 依赖注入:

```
package edu.nju.demo.controller;

@RestController
@RequestMapping("/api/recharges")
public class RechargeController {
    private final RechargeService rechargeService;

    public RechargeController(RechargeService rechargeService) {
        this.rechargeService = rechargeService;
    }

    @PostMapping
    public RechargeResult recharge(@RequestBody RechargeCommand command) {
        return rechargeService.recharge(command);
    }
}
```

1.7 六、详细设计题模板

1.7.1 Controller-Service-Repository 顺序图

```

User -> Controller: HTTP request
Controller -> Service: business method
Service -> Repository: query/save
Repository -> Database: SQL
Database --> Repository: rows
Repository --> Service: entity
Service --> Controller: result DTO
Controller --> User: HTTP response

```

1.7.2 控制风格

集中式控制：控制对象掌握大量流程，流程清楚，但容易臃肿。

分散式控制：多个对象共同推进流程，单对象轻，但控制流难追踪。

委托式控制：入口对象保留主流程，子任务委托给专业对象，是业务系统中常用方案。

情境判断：

充值、排课、采购补货这类流程包含校验、计算、持久化和外部接口调用，适合委托式控制：Controller 接收请求，Service

1.7.3 GRASP

Information Expert：把责任交给拥有信息的对象。

Creator：由包含、记录或紧密使用目标对象的类创建目标对象。

Controller：由系统事件入口对象协调业务流程。

Low Coupling：减少不必要依赖。

High Cohesion：让类职责集中。

1.8 七、模块化与原则题模板

耦合：

数据耦合：只通过必要数据参数交互。

印记耦合：传递整个对象，但只使用其中一部分。

控制耦合：传递标志或类型，让被调用者选择行为。

公共耦合：共享全局数据。

内容耦合：直接访问或修改对方内部。

内聚：

功能内聚：模块只完成一个明确功能。

信息内聚：围绕同一数据结构组织多个操作。

通信内聚：操作同一数据集合。

过程内聚：按流程顺序组织，但职责不单一。

逻辑内聚：根据类型参数选择不同逻辑。

偶然内聚：职责无明确关联。

原则对应：

SRP：一个模块只有一个变化理由。
OCP：对扩展开放，对修改关闭。
DIP：高层和低层都依赖抽象。
ISP：接口小而专用。
LSP：子类能替换父类。
Law of Demeter：只与直接朋友通信。
组合优于继承：用组合和委托降低白盒复用风险。

1.9 八、设计模式模板

1.9.1 Strategy

适用：同一业务存在多种算法，如支付方式、优惠券、冲突检测、订货规则。

```
Context -> Strategy
Strategy <|.. ConcreteStrategyA
Strategy <|.. ConcreteStrategyB
```

答案句：

把变化的算法抽象成 **Strategy** 接口，**Context** 只依赖接口。新增算法时新增实现类，不修改核心流程，符合 OCP 和 DIP。

1.9.2 Abstract Factory / DAO

适用：一组相关产品或数据访问实现需要整体替换，如 MySQL DAO 与 SQLServer DAO。

```
DaoFactory -> UserDao
DaoFactory -> OrderDao
MySQLDaoFactory creates MySQLUserDao, MySQLOrderDao
SqlServerDaoFactory creates SqlServerUserDao, SqlServerOrderDao
```

1.9.3 Observer

适用：一个对象状态变化后通知多个显示或订阅对象。

```
Subject attach/detach/notify
Observer update
ConcreteObserver receive update
```

1.9.4 Iterator

适用：遍历集合而不暴露内部结构。

1.10 九、软件构造题模板

代码坏味道：

- 命名无意义。
- 方法过长。
- 条件表达式复杂。
- 魔法数字。
- 重复代码。
- 职责混杂。
- 缺少异常处理。
- 资源关闭不可靠。
- 参数过多。
- 直接依赖具体实现。

改造方向：

- 提取方法。
- 引入有意义命名。
- 使用表驱动或策略模式消除长分支。
- 把业务规则放入 **Service** 或领域对象。
- 使用 **guard clause** 处理非法输入。
- 使用 **try-with-resources** 管理资源。
- 用常量或配置表达业务阈值。
- 补充单元测试。

表驱动例子：

```
private static final NavigableMap<Integer, Integer> RULES = new TreeMap<>();
static {
    RULES.put(3, 7);
    RULES.put(7, 15);
    RULES.put(10, 20);
    RULES.put(15, 30);
    RULES.put(Integer.MAX_VALUE, 40);
}

public int stockDays(int leadDays) {
    if (leadDays < 0) {
        throw new IllegalArgumentException("leadDays");
    }
    return RULES.higherEntry(leadDays).getValue();
}
```

1.11 十、测试题模板

1.11.1 圈复杂度

$V(G) = \text{判定节点数} + 1$

判定节点包括：**if**、**while**、**for**、**case**、复杂布尔表达式中的独立条件按题目要求处理。

1.11.2 黑盒测试

等价类划分：有效类与无效类。

边界值分析：边界、边界内、边界外。

决策表：多个条件组合决定多个动作。

状态转换：对象状态随事件改变。

1.11.3 白盒测试

语句覆盖：每条语句至少执行一次。

分支覆盖：每个判定的真分支和假分支至少执行一次。

条件覆盖：每个基本条件的真值和假值至少出现一次。

路径覆盖：覆盖可行路径。

测试用例表格式：

编号 | 输入 | 覆盖点 | 预期输出

T1 | amount=100, card=normal | 正常路径 | 成功充值

T2 | amount=0 | 金额边界 | 提示金额非法

T3 | card=lost | 卡状态异常 | 拒绝充值

1.12 十一、HCI 题模板

答题四步：

1. 描述界面现象。
2. 对应 HCI 原则。
3. 分析对用户任务的影响。
4. 给出改进建议。

常用原则：

系统状态可见。

与现实世界匹配。

用户控制和自由。

一致性和标准。

错误预防。

识别优于回忆。

灵活性和效率。

美学和极简设计。

帮助用户识别、诊断和恢复错误。

帮助和文档。

Fitts 定律。

及时反馈。

导航清晰。

1.13 十二、最后背诵清单

软件工程定义

SCM 六活动和变更控制六步

需求三层次和六类软件需求

用例四要素、**include/extend**

概念类图六步

Web 物理包图固定结构

Controller/Service/Repository 职责和依赖注入代码

三种控制风格

耦合、内聚、**SOLID**、信息隐藏

Strategy、**Abstract Factory**、**Observer**、**Iterator**

代码坏味道和表驱动

圈复杂度、边界值、条件覆盖、分支覆盖

HCI 十项启发式